Learning R for Data Science & Data Analysis

Self-Paced Learning Journey

2026-11-05

import libraries that help to create a database connection

```
library(DBI)
library(RPostgres)
```

```
# 1. Connect to PostgreSQL database (must already exist)
# Define your connection parameters
db_host     <- "localhost"
db_port     <- 5432
db_name     <- "learning_r_database"
db_user     <- "postgres"
db_password <- "07288"
```

```
# Safely attempt to connect
con <- tryCatch({
  connection <- dbConnect(
    RPostgres::Postgres(),
    host     = db_host,
    port     = db_port,
    dbname   = db_name,
    user     = db_user,
    password = db_password
  )

  # If successful, print a clear visual confirmation
  cat("[STATUS] Database connected successfully! n")
  cat("Connected to:", db_name, "on host:", db_host, "\n\n")

  # Return the connection object to the variable 'con'
  connection

}, error = function(e) {
  # If an error happens, print a failure message instead of crashing
  cat("[STATUS] Database is NOT connected. n")
  cat("Error details:", e$message, "\n\n")

  # Return NULL so 'con' is explicitly empty
  NULL
})
```

```
## [STATUS] Database connected successfully! nConnected to: learning_r_database on host: localhost
```

```
# 2. Create the 3 relational tables
dbExecute(con, "
  CREATE TABLE IF NOT EXISTS student (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    enrollment_year INT NOT NULL
  );
")
```

```
## NOTICE:  relation "student" already exists, skipping
```

```
## [1] 0
```

```
dbExecute(con, "
  CREATE TABLE IF NOT EXISTS course (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT NOT NULL
  );
")
```

```
## NOTICE:  relation "course" already exists, skipping
```

```
## [1] 0
```

```
dbExecute(con, "
  CREATE TABLE IF NOT EXISTS marks (
    mark_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES student(student_id) ON DELETE CASCADE,
    course_id INT REFERENCES course(course_id) ON DELETE CASCADE,
    score INT NOT NULL CHECK (score BETWEEN 0 AND 100)
  );
")
```

```
## NOTICE:  relation "marks" already exists, skipping
```

```
## [1] 0
```

3. Insert base lookup data (Students and Courses)

```
# Generate mock student names
first_names <- c("John", "Mary", "David", "Sarah", "James", "Elena", "Robert", "Linda", "Michael", "Pat
last_names <- c("Smith", "Jones", "Miller", "Davis", "Garcia", "Rodriguez", "Wilson", "Thomas", "Martin

set.seed(42) # For reproducible random data
mock_students <- data.frame(
```

```
  name = paste(sample(first_names, 40, replace = TRUE), sample(last_names, 40, replace = TRUE)),
  enrollment_year = sample(2023:2026, 40, replace = TRUE)
)

mock_courses <- data.frame(
  course_name = c("Introduction to R", "Database Systems", "Linear Models", "Data Structures", "Machine
  credits = c(3, 4, 3, 4, 4)
)
```

```
# Write students and courses to DB
dbWriteTable(con, "student", mock_students, append = TRUE, row.names = FALSE)
dbWriteTable(con, "course", mock_courses, append = TRUE, row.names = FALSE)
```

4. Generate and insert 200 random mark records linking them

```
# Fetch the generated database IDs to avoid foreign key violations
student_ids <- dbGetQuery(con, "SELECT student_id FROM student;")$student_id
course_ids <- dbGetQuery(con, "SELECT course_id FROM course;")$course_id

mock_marks <- data.frame(
  student_id = sample(student_ids, 200, replace = TRUE),
  course_id = sample(course_ids, 200, replace = TRUE),
  score = sample(50:100, 200, replace = TRUE) # Scores between 50 and 100
)
```

```
# Write the 200 records to the marks table
dbWriteTable(con, "marks", mock_marks, append = TRUE, row.names = FALSE)
```

```
cat("Database setup complete! Created tables and inserted 200 mock records.\n")
```

```
## Database setup complete! Created tables and inserted 200 mock records.
```

```
# Check if tables created
print(dbListTables(con))
```

```
## [1] "course"  "marks"   "student"
```

```
# Calling database tables into R
course_df <- dbReadTable(con, "course")
(head(course_df))
```

```
##   course_id       course_name credits
## 1         1 Introduction to R       3
## 2         2  Database Systems       4
## 3         3     Linear Models       3
## 4         4   Data Structures       4
## 5         5  Machine Learning       4
## 6         6 Introduction to R       3
```

```r
student_df <- dbReadTable(con, "student")
(head(student_df))
```

```
##   student_id            name enrollment_year
## 1          1      John Jones            2023
## 2          2    James Thomas            2024
## 3          3      John Jones            2023
## 4          4  Michael Miller            2024
## 5          5 Patricia Thomas            2026
## 6          6    Sarah Wilson            2025
```

```r
marks_df <- dbReadTable(con, "marks")
(tail(marks_df))
```

```
##      mark_id student_id course_id score
## 1795    1795        307        43    63
## 1796    1796         10        45    81
## 1797    1797        121        35    77
## 1798    1798         89        42    92
## 1799    1799         37        34    81
## 1800    1800        194        39    90
```

Accessing Built-in Datasets

```r
# Load the historic 'mtcars' dataset (Motor Trend Car Road Tests)
data("mtcars")

# Convert it explicitly into a clean data frame structure
cars_df <- as.data.frame(mtcars)

# Print structural diagonistics
class(cars_df)
```

```
## [1] "data.frame"
```

```r
dim(cars_df)  # Display total number of rows and columns
```

```
## [1] 32 11
```

```r
head(cars_df, n=3) # Display only first 3 rows
```

```
##                mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4     21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710    22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

6. Ingesting Real-Time Data via REST APIs

In production environments, data scientist frequently pull live web data via REST APIs using HTTP requests.

We utilize the `httr2` library to handle the the requests and `jsonlite` to convert the raw JSON web response into a clean R data frame.

```
# 1. Clear any cached bad text response from R's global memory
if (exists("raw_response")) rm(raw_response)

# 2. Load necessary packages
library(httr2)
library(jsonlite)

# 3. Define the precise API endpoint (Double-check the "open." prefix)
api_url <- "https://open.er-api.com/v6/latest/USD"

# 4. Perform the HTTP request and stream the body text
raw_response <- request(api_url) |>
  req_perform() |>
  resp_body_string()

# 5. Diagnostic Check: Print the first 50 characters to verify it is NOT HTML
cat("Memory Check (First 50 chars):", substr(raw_response, 1, 50), "\n\n")
```

```
## Memory Check (First 50 chars): {"result":"success","provider":"https://www.exchan
```

```
# 6. Parse the validated JSON string into a structured data frame
parsed_list <- jsonlite::fromJSON(raw_response)
rates_df <- data.frame(
  currency = names(parsed_list$rates),
  rate_to_usd = as.numeric(parsed_list$rates)
)

# 7. Print the final clean dataset rows
head(rates_df, 5)
```

```
##   currency rate_to_usd
## 1      USD     1.00000
## 2      AED     3.67250
## 3      AFN    64.83332
## 4      ALL    81.08182
## 5      AMD   360.86220
```

Module 1.3: Indexing & Subsetting Data

Extracting targeted subsets of data is essential for filtering records before executing visualizations or statistical models. In base R, we use two primary methods to isolate data: **The Dollar Sign Operator (`$`)** for single columns, and **Square Brackets (`[row, column]`)** for multi-dimensional slicing.

1. The Dollar Sign Operator ($)

- **Purpose:** Extracts a single column out of a data frame and converts it into a flat, one-dimensional vector.
- **Syntax:** `dataframe_name$column_name`

2. Square Bracket Matrix Notation ([,])

- **Purpose:** Slices data frames across two dimensions simultaneously using index numbers or coordinates.
- **Syntax:** `dataframe_name[row_index, column_index]`
- **The Blank Rule:** Leaving a side blank selects *everything* in that dimension. `[1, ]` means "Row 1, all columns." `[ , 2]` means "All rows, Column 2."

```
# -------1. Extracting Singlr Vectors via the Dollar Sign ($) ------------
# Extract the score colun from the SQR marks data frame
all_scores <- marks_df$score
head(all_scores, 10)
```

```
##  [1] 66 99 88 94 83 61 79 84 69 75
```

```
# -------2. Coordinate slicing via Square Brackets ([,]) ----------------

#Example A: Extract the value from Row 3, Column 2 of the database courses
specific_course <- course_df[3, 2]
print(specific_course)
```

```
## [1] "Linear Models"
```

```
# Example B: Extract first 5 rows of ALL columns from the cars dataset
cars_slice <- cars_df[1:5, ]
print(cars_slice)
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
```

```
# Example C: Extract rows 10 through 15 of only column 1 from the exchange rates
rate_slice <- rates_df[10, 15]
print(rate_slice)
```

```
## NULL
```

Visual Function Mapping Index

Plots are grouped by analytical application rather than function names to streamline code deployment:

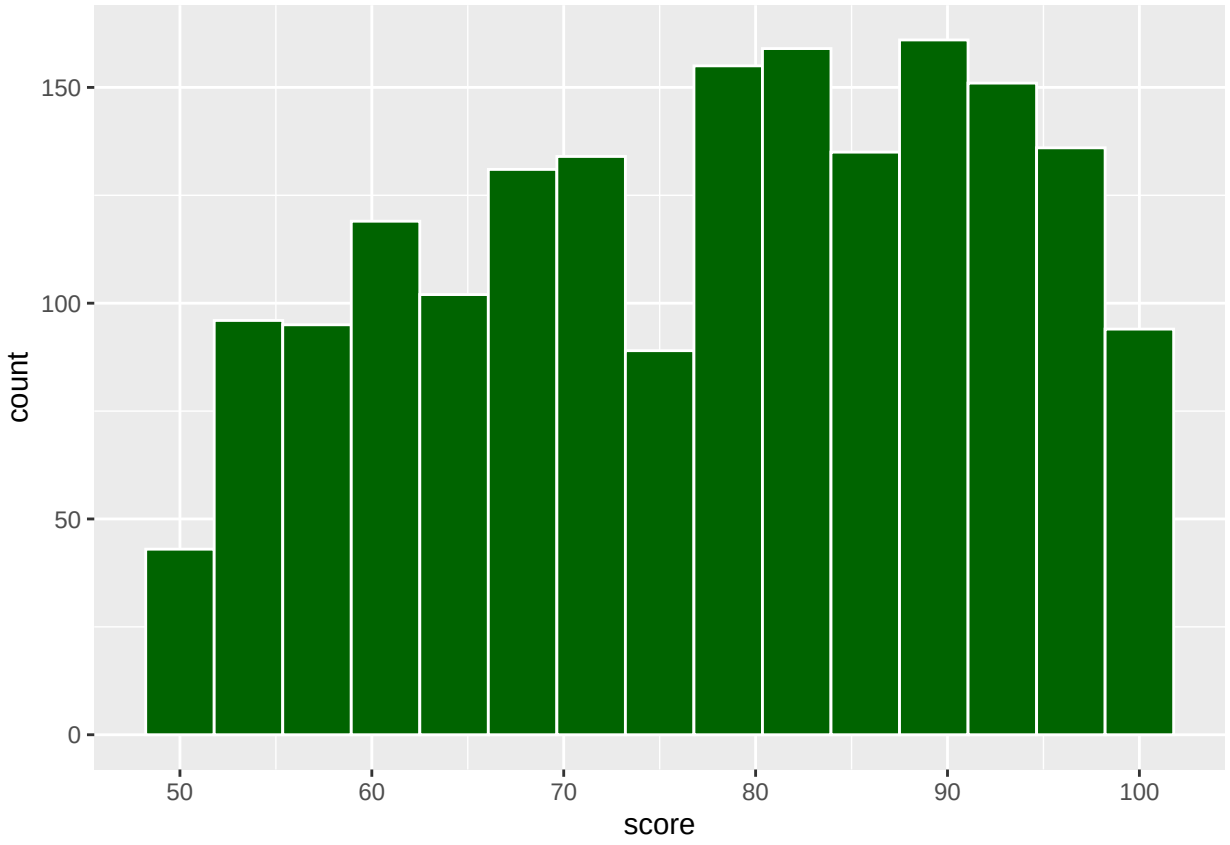
Group A: Distribution Analytics (Single Continuous Variable)

- `geom_histogram()`: Continuous block distribution.
- `geom_density()`: Continuous smoothed density lines.
- `geom_rug()`: Marginal axis distribution tick marks.

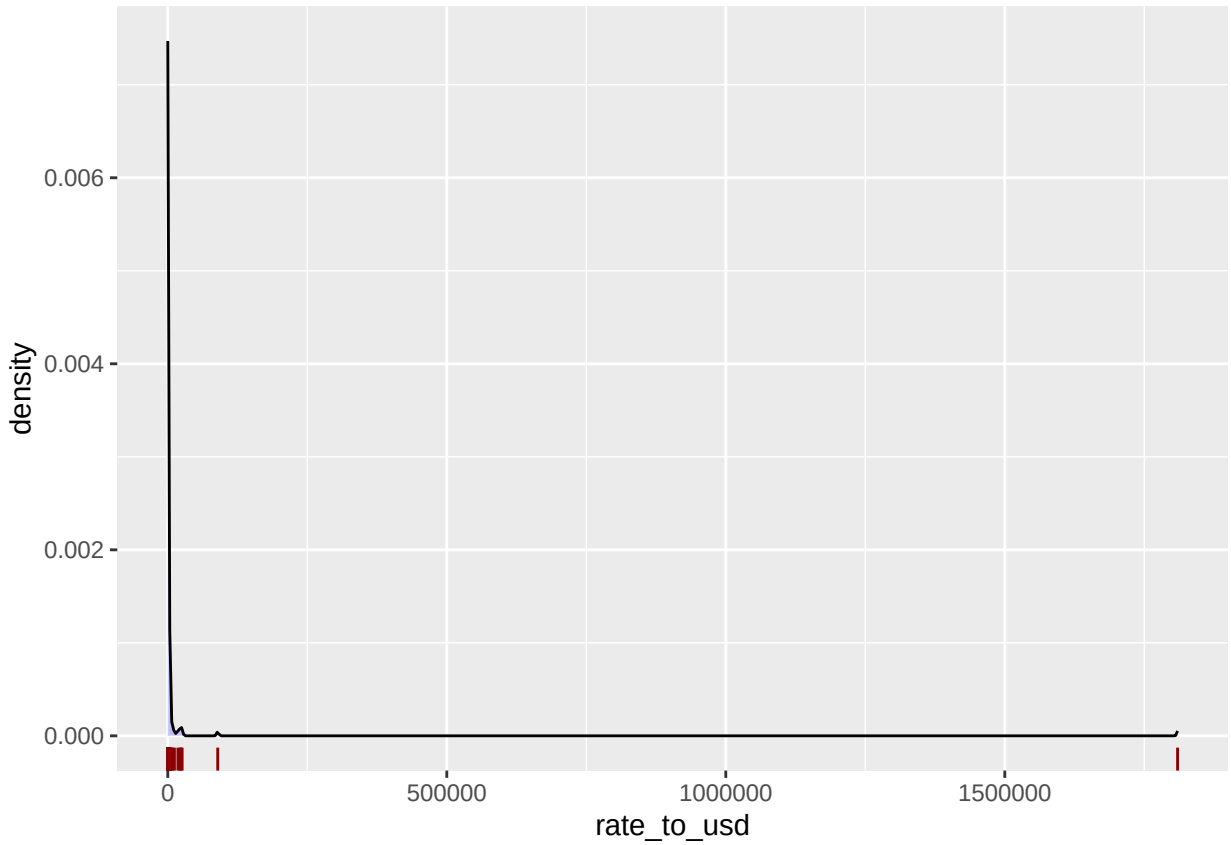
Application of Group A: We use these functions to evaluate the spread and shape of a single numeric column. `geom_histogram()` splits data into absolute bins, `geom_density()` calculates a smooth probability wave, and `geom_rug()` adds marginal tick marks to show exact data concentration points.

```
library(ggplot2)

# 1. Histogram displaying student scores from the database
ggplot(data = marks_df, aes(x = score)) +
  geom_histogram(bins = 15, fill = "darkgreen", color = "white")
```




```
# 2. Smooth density line paired with a marginal rug plot using API data
ggplot(data = rates_df, aes(x = rate_to_usd)) +
  geom_density(fill = "blue", alpha = 0.2) +
  geom_rug(color = "darkred")
```

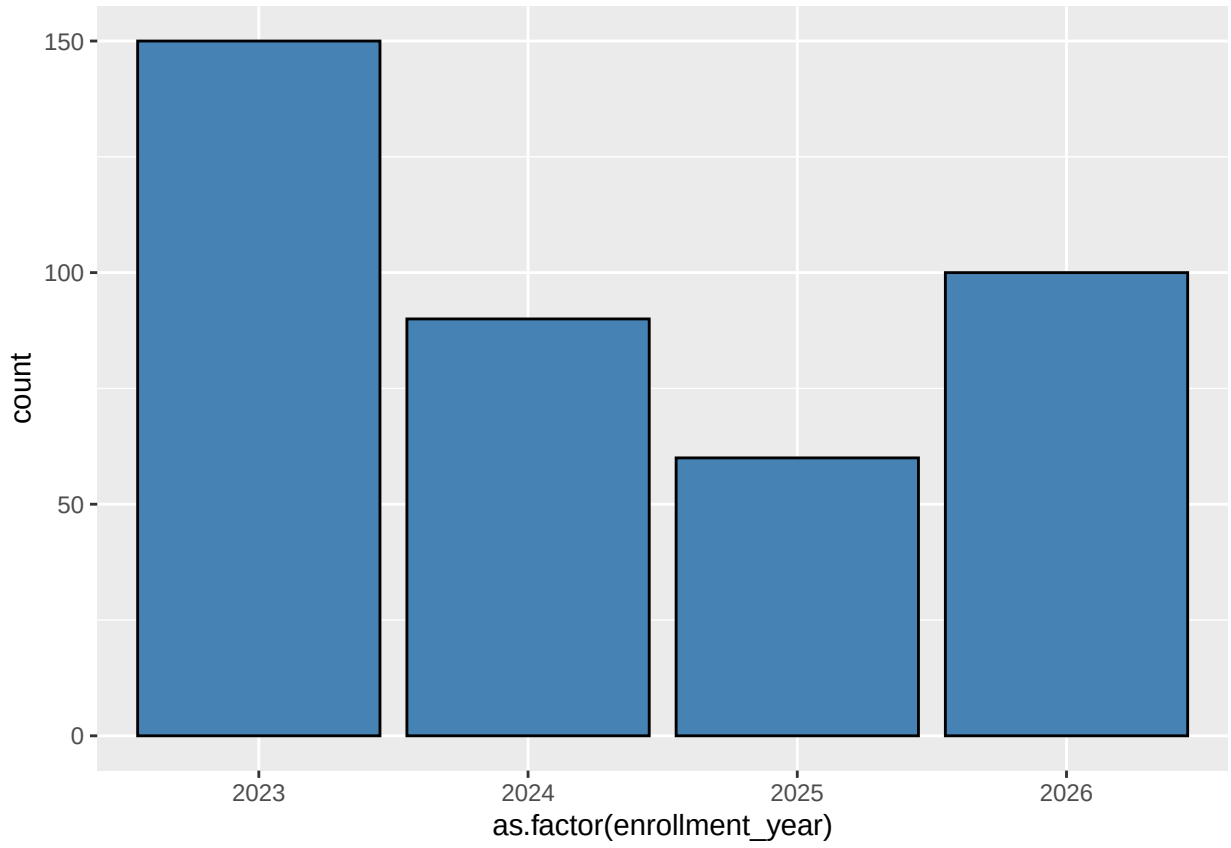


Group B: Categorical & Comparison Analytics

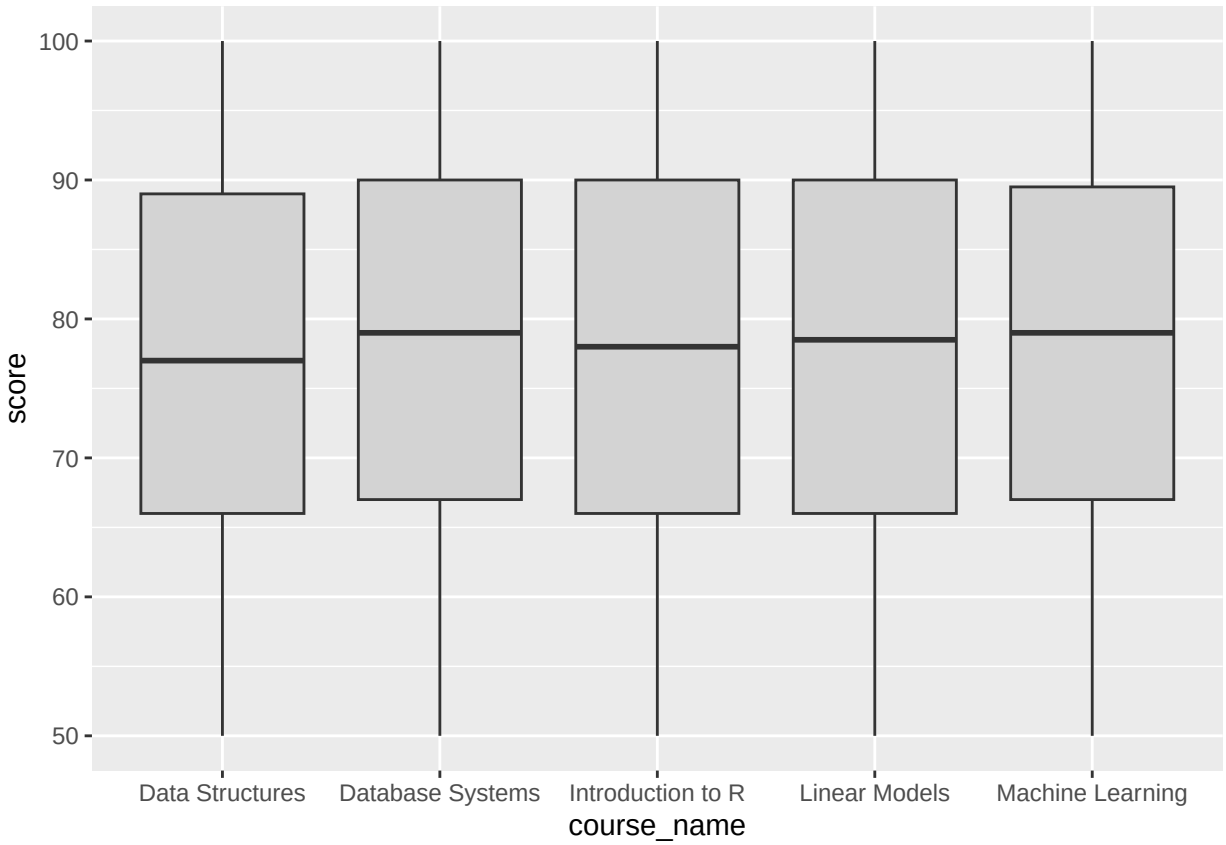
- `geom_bar()`: Frequency tracking across absolute categories.
- `geom_boxplot()`: Strategic metric variation across groups.
- `geom_violin()`: Dual density profile variation across groups.

Application of Group B: We use these functions to compare numeric metrics across distinct categories or to count categorical frequencies. `as.factor()` is applied to numeric columns (like years or cylinder counts) to force R to treat them as separate comparative groups.

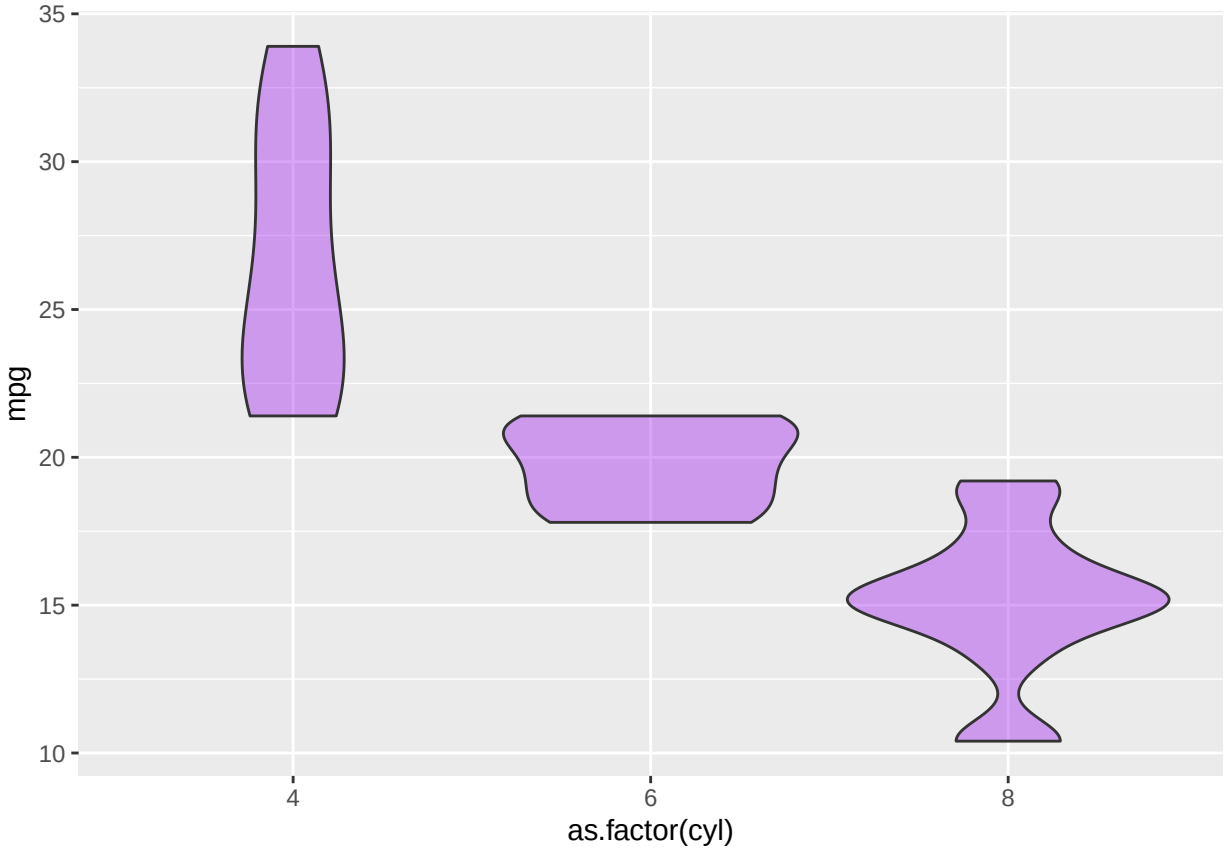
```r
# 1.  Bar Chart counting rows per category in database student table
ggplot(data = student_df, aes(x = as.factor(enrollment_year))) +
  geom_bar(fill = "steelblue", color = "black")
```




```r
# 2.  Boxplot displaying database scores across different course names
merged_marks <- merge(marks_df, course_df, by = "course_id")
ggplot(data = merged_marks, aes(x = course_name, y = score)) +
  geom_boxplot(fill = "lightgray")
```




```
# 3.  Violin plot displaying the density shape of a car MPG by engine cylinder count
ggplot(data = cars_df, aes(x = as.factor(cyl), y = mpg)) +
  geom_violin(fill = "purple", alpha = 0.4)
```



Group C: Correlation Analytics (Two Continuous Variables)

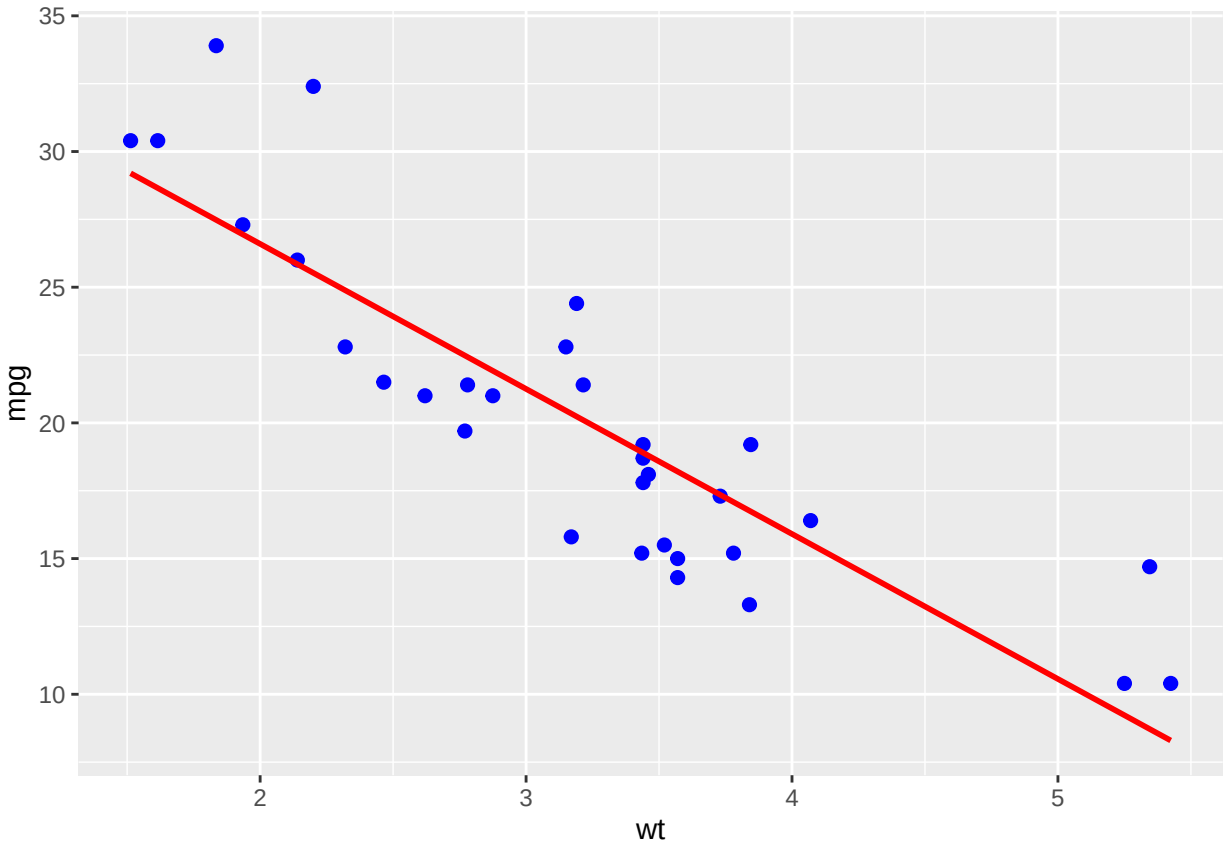
- `geom_point()`: Standard bivariate scatter arrays.
- `geom_jitter()`: Scattered overlay arrays to counter data overlap.
- `geom_line()`: Sequential connected data tracking.
- `geom_smooth()`: Conditional local trend and regression fits.

Application of Group C: We use these functions to evaluate relationships between two numeric columns. `geom_point()` maps exact coordinates, `geom_jitter()` prevents overlapping points from hiding data density, and `geom_smooth()` fits trend lines.

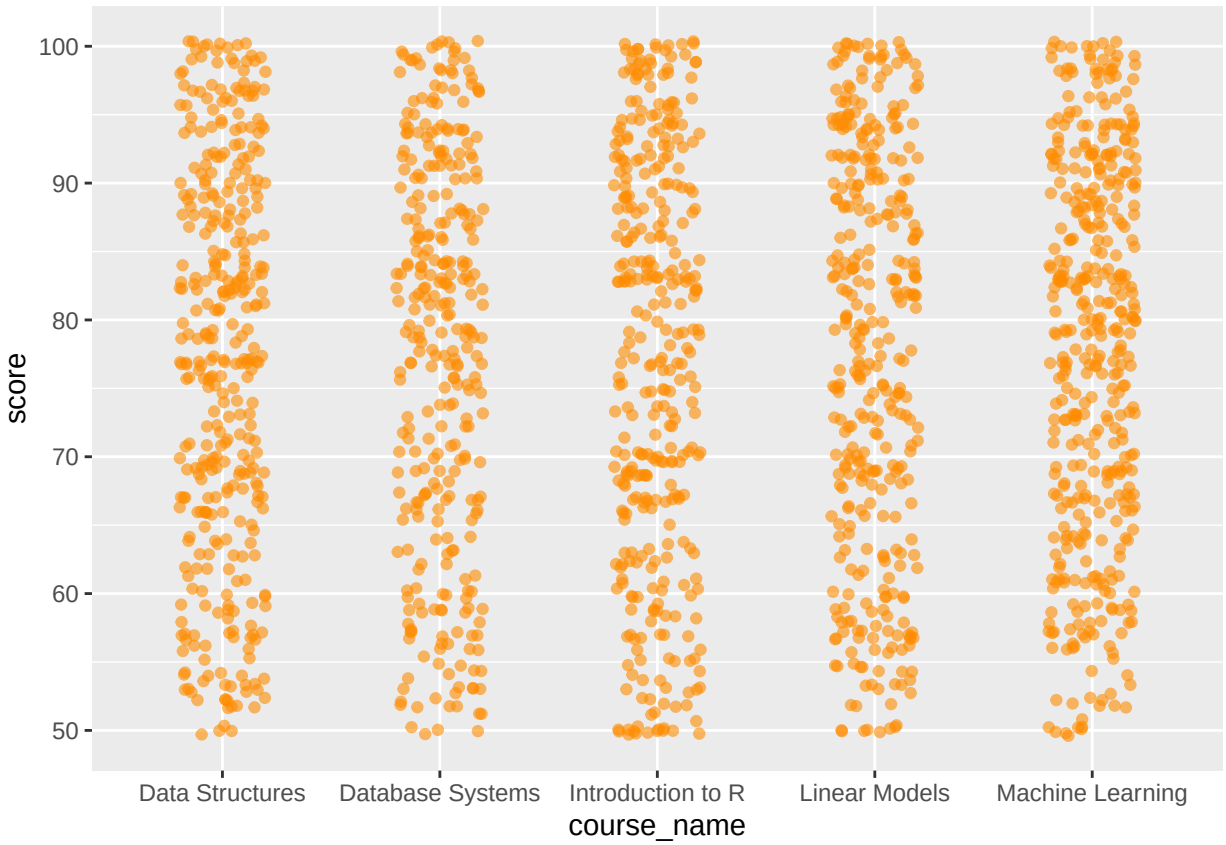
```
library(ggplot2)

# 1.  Scatter plot combining points and linear trend line using car data
ggplot(data = cars_df, aes(x = wt, y = mpg)) +
  geom_point(color = "blue", size = 2) +
  geom_smooth(method = "lm", color = "red", se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
# 2.  Jitter plot showing dense database score points across categories without stacking
ggplot(data = merged_marks, aes(x = course_name, y = score)) +
  geom_jitter(color = "darkorange", width = 0.2, alpha = 0.6)
```



Group D: Structural Annotation Overlays

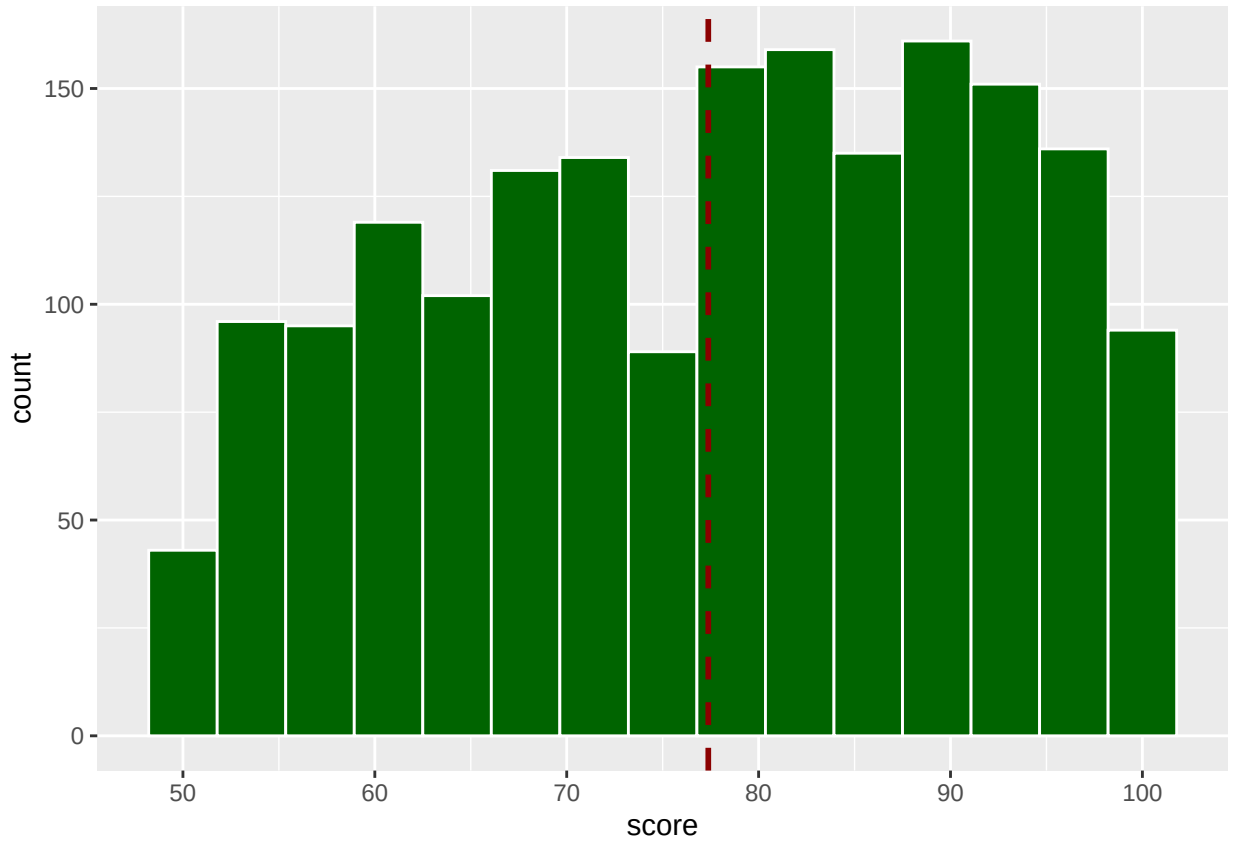
- `geom_text()`: Vector-mapped textual identifiers.
- `geom_hline()`: Constant horizontal boundaries.
- `geom_vline()`: Constant vertical thresholds.

Application of Group D: We use these functions to draw reference guides directly onto active charts. `geom_hline()` and `geom_vline()` require manual intercept definitions, while `geom_text()` prints character data directly inside the graph area.

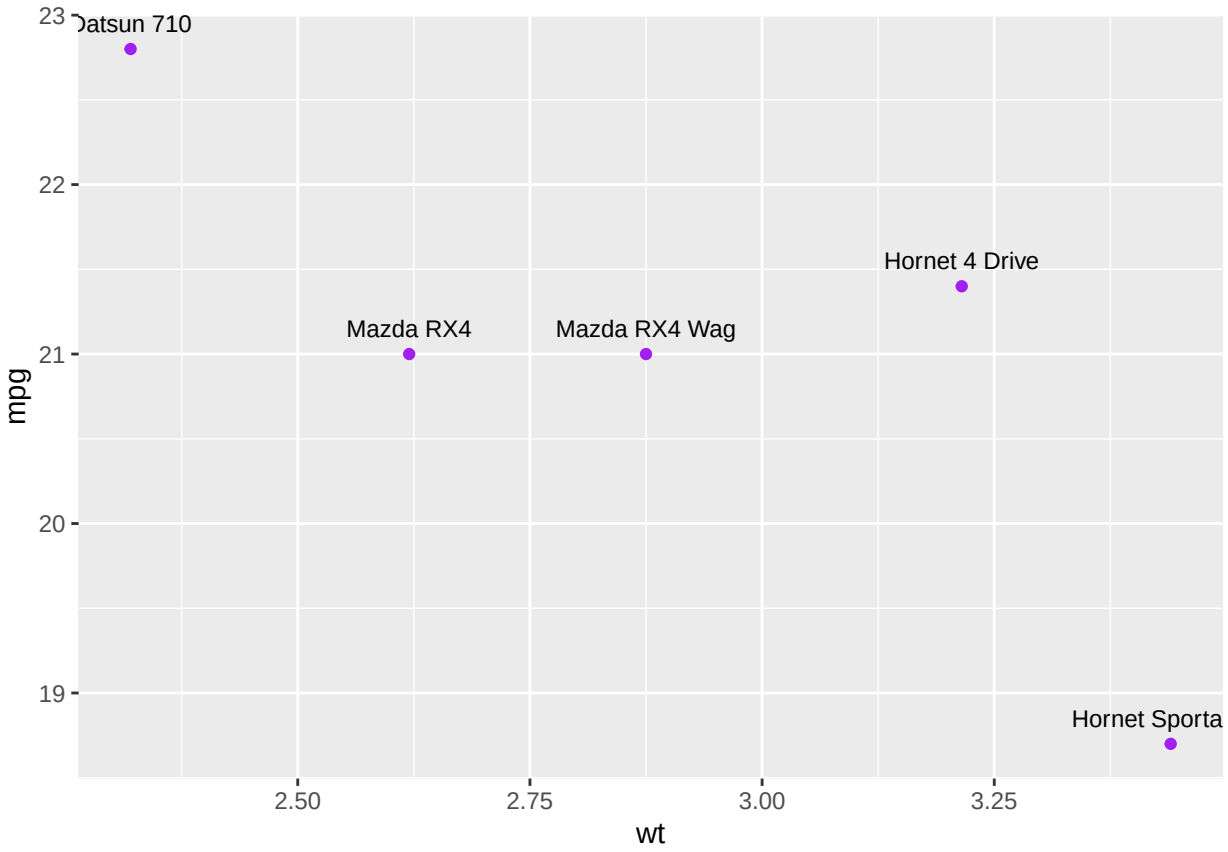
```
# Calculate statistical reference points
avg_score <- mean(marks_df$score)

# Histogram with a vertical reference line cutting through the calculated mean score
ggplot(data = marks_df, aes(x = score)) +
  geom_histogram(bins = 15, fill = "darkgreen", color = "white") +
  geom_vline(xintercept = avg_score, color = "darkred", linetype = "dashed", size = 1)
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## This warning is displayed once per session.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```




```
# Scatter plot with text annotations mapping the first 5 car models to their coordinates
ggplot(data = cars_df[1:5, ], aes(x = wt, y = mpg)) +
  geom_point(color = "purple") +
  geom_text(aes(label = rownames(cars_df[1:5, ])), vjust = -1, size = 3)
```



Group E: Visual Standardization & Themes (The Finishing Touch)

- `theme_minimal()`: Removes heavy grey backgrounds for a clean, modern look.
- `theme_classic()`: Removes gridlines completely, leaving only the X and Y axes.
- `labs()`: Adds professional titles, subtitles, captions, and axis labels.

Application of Group E: No chart is ready to be shared with a team or posted on GitHub without clear labels and a clean theme. We layer `theme_minimal()` and `labs()` onto any plot to instantly change it from a rough draft into a polished, professional visualization.

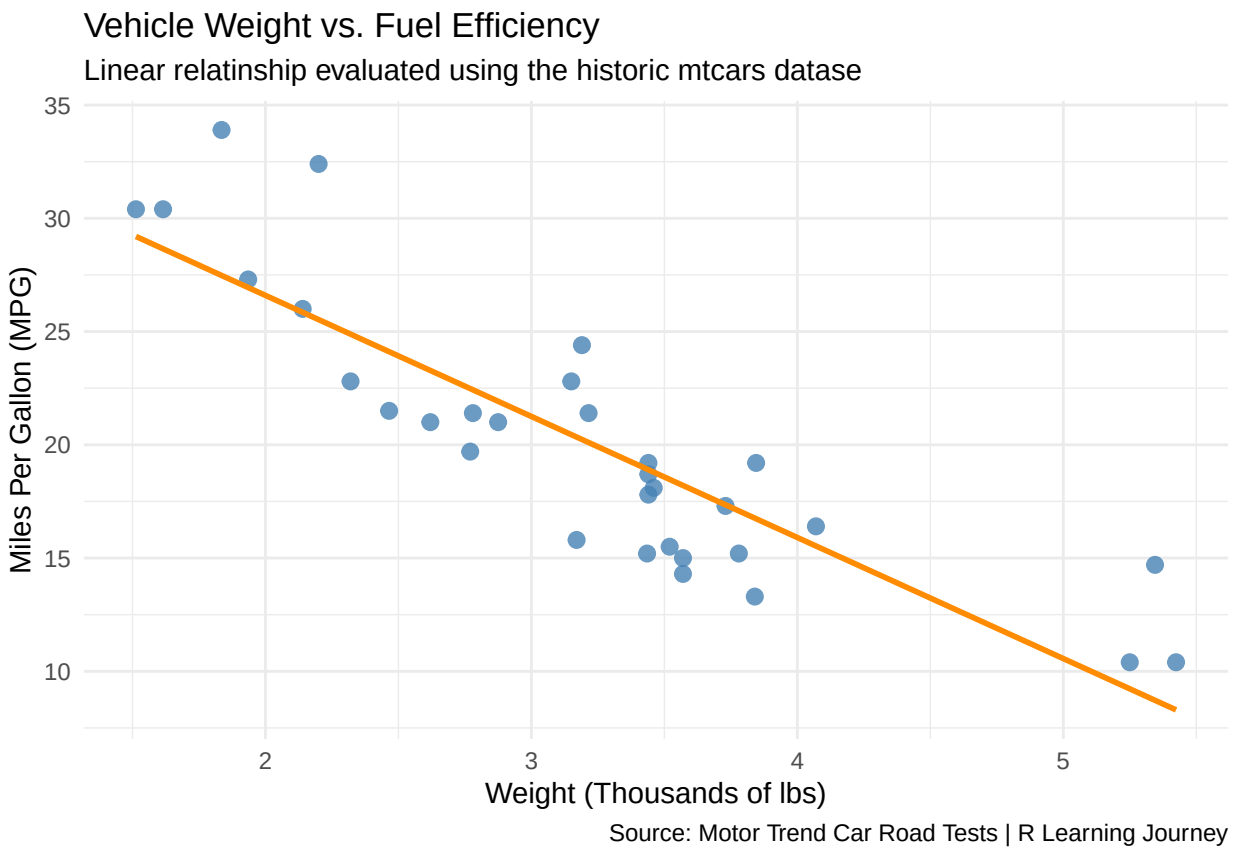
```
# Take the original car scatter plot and apply professional design standards
ggplot(data = cars_df, aes(x = wt, y = mpg)) +
  geom_point(color = "steelblue", size = 2.5, alpha = 0.8) +
  geom_smooth(method = "lm", colour = "darkorange", se = FALSE, size = 1) +

  # Apply a clean, modern presentation theme
  theme_minimal() +

  # Explicitly label every component of the visualization
  labs(
    title = "Vehicle Weight vs. Fuel Efficiency",
    subtitle = "Linear relatinship evaluated using the historic mtcars datase",
    x = "Weight (Thousands of lbs)",
```

```
    y = "Miles Per Gallon (MPG)",
    caption = "Source: Motor Trend Car Road Tests | R Learning Journey"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Phase 2.1: Data Wrangling with `dplyr`

Data wrangling involves filtering rows, selecting columns, creating new attributes, and sorting observations. We will utilize three fundamental functions to isolate precise subsets of our data: * `filter()`: Extracts rows based on a logical condition (e.g., scores > 80). * `select()`: Isolates specific columns by name, removing unnecessary variables. * `arrange()`: Sorts rows based on values in a specified column (ascending or descending).

1. Conditional Row Extraction: `filter()`

The `filter()` function checks a column and keeps only the rows where our condition evaluates to `TRUE`. We chain this immediately after our dataset using the native pipe operator `|>`.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# Isolate all records from the marks database table where the score is 90 or above
high_score_df <- marks_df |>
  filter(score >= 90)

# Print structural metrics to verify the row filtering
dim(marks_df) # Original row count (200)
```

```
## [1] 1800    4
```

```
dim(high_score_df)  # New filtered row count
```

```
## [1] 460   4
```

```
head(high_score_df, 5)
```

```
##   mark_id student_id course_id score
## 1       2          5         2    99
## 2       4         16         1    94
## 3      15         31         1    95
## 4      17         30         5    95
## 5      19          6         4    99
```

2. Column Selection and Sorting: `select()` and `arrange()`

We string multiple operations together in a single pipeline. First, we filter the data; second, we use `select()` to keep only the student ID and score columns; third, we use `arrange(desc())` to sort the list with the highest score at the very top.

```
# Build a sorted leaderboard showing only specific columns
markes_leaderboard <- marks_df |>
  filter(score >= 85) |>
  select(student_id, score) |>
  arrange(desc(score))


# Print the top 5 records of the processed leaderboard
head(markes_leaderboard)
```

```
##   student_id score
## 1          2   100
## 2         28   100
## 3         35   100
## 4         31   100
## 5         24   100
## 6          7   100
```

Phase 2.2: User-Defined Functions & Functional Programming

1. Predefined vs. User-Defined Functions (UDFs)

A **predefined function** is built into R (e.g., `mean()`, `class()`). A **user-defined function** is a custom pipeline you build to execute repetitive tasks.

A function structure contains three parts: 1. **Arguments:** The inputs passed into your function. 2. **Body:** The actual operations executed behind the scenes. 3. **Return Statement:** The final output sent back to your environment.

```
# Define a custom function to classify scores into Pass/Fail categories
classify_performance <- function(student_score){
  if (student_score >= 70){
    return("Pass")
  }else{
    return("Fail")
  }
}

# Test new function on a standalone value
classify_performance(85)
```

```
## [1] "Pass"
```

```
classify_performance(56)
```

```
## [1] "Fail"
```

2. The Functional Iteration Family (`lapply`, `sapply`, `tapply`)

Loops can be slow and hard to read in data science. R uses the `apply` family to map a function over elements of an object efficiently.

- `lapply()`: Loops over a vector or list and returns a **List**.
- `sapply()`: Loops over an object and simplifies the output into a flat **Vector**.
- `tapply()`: Computes a function on a continuous variable **grouped by a categorical factor** (highly useful for group summaries).

```r
# A.  Vector to vector Mapping with spply()
# Take the first 5 scores from the database and check their pass/fail status
sample_scores <- head(marks_df$score, 5)
pass_fail_vector <- sapply(sample_scores, classify_performance)
print(pass_fail_vector)
```

```
## [1] "Fail" "Pass" "Pass" "Pass" "Pass"
```

```r
# B.  Vector to List Mapping with lapply
pass_fail_list <- lapply(sample_scores, classify_performance)
print(pass_fail_list)
```

```
## [[1]]
## [1] "Fail"
##
## [[2]]
## [1] "Pass"
##
## [[3]]
## [1] "Pass"
##
## [[4]]
## [1] "Pass"
##
## [[5]]
## [1] "Pass"
```

```r
# C.  Grouped Aggregation with tapply
# Calculate the average database score GROUPED by each course ID
average_by_course <- tapply(marks_df$score, marks_df$course_id, mean)
print(average_by_course)
```

```
##        1        2        3        4        5        6        7        8
## 77.83333 80.84783 79.71028 77.76543 76.98876 74.96923 79.21739 74.94872
##        9       10       11       12       13       14       15       16
## 74.87719 77.49580 79.93548 74.67347 77.98039 79.17544 78.75000 74.66667
##       17       18       19       20       21       22       23       24
## 72.50000 80.80488 76.75439 77.36207 75.97297 71.06897 77.19048 72.40909
##       25       26       27       28       29       30       31       32
## 77.00000 76.20833 77.83871 72.16667 83.41667 80.75000 74.88889 83.18182
##       33       34       35       36       37       38       39       40
## 70.40000 72.00000 75.71429 84.12500 77.18182 76.40000 80.42105 84.33333
##       41       42       43       44       45       46       47       48
## 76.25000 80.20000 78.10000 75.75000 78.94444 79.40000 80.33333 74.00000
##       49       50
## 84.50000 54.00000
```

3. Advanced UDF: Automated Statistical Testing (Two-Sample t-Test)

A **Two-Sample t-Test** evaluates whether the means of two independent numeric groups are statistically different from each other.

Let's build a complex user-defined function. It will take a data frame, extract two specific categorical groups, execute a predefined `t.test()`, and extract a clean statement about whether the difference is statistically significant (based on a p-value threshold of 0.05).

```
# Define a robust function to compare car fuel effiencies across 4 vs 6 cylinder engines
run_custom_ttest <- function(dataset, continuous_var, categorical_var, group_a, group_b){
  # Step A: Filter out the two targeted comparion groups using matrix indexing
  data_group_1 <- dataset[dataset[[categorical_var]] == group_a, continuous_var]
  data_group_2 <- dataset[dataset[[categorical_var]] == group_b, continuous_var]

  #Step B: Run R's predefined t-test function
  test_result <- t.test(data_group_1, data_group_2)

  # Step C: Extract the p-value
  extracted_p_value <- test_result$p.value

  # Step D: Apply conditional formatting to deliver a clear plain-text answer
  if (extracted_p_value < 0.05){
    conclusion <- "Significant DIfference: The groups are statistically distinct"
  }else{
    conclusion = "No Significant Diffence: Observed variations are likely due to chance"
  }

  # Return a comprehnsive list summary of our results
  return(list(
    p_value = extracted_p_value,
    evaluation = conclusion
  ))
}

# Run the custom function on the mtcars dataset comparing 4-cylinder vs 6-cylinder cars
car_analysis <- run_custom_ttest(
  dataset = cars_df,
  continuous_var = "mpg",
  categorical_var = "cyl",
  group_a = 4,
  group_b = 6
)

print(car_analysis)
```

```
## $p_value
## [1] 0.0004048495
##
## $evaluation
## [1] "Significant DIfference: The groups are statistically distinct"
```

4. Multivariate Functional Iteration: `mapply()`

While `sapply()` and `lapply()` are limited to looping over a single vector at a time, `mapply()` handles multivariate tracking. It passes elements from **multiple vectors simultaneously** into a matching multi-argument user-defined function.

```r
# Define a custom function requiring two distinct criteria inputs
flag_student_performance <- function(score, year){
  if (score >= 90 & year == 2024){
    return("Elite (2024 cohort)")
  }else if (score >= 85){
    return("High Performer")
  }else {
    return("Standard Performance")
  }
}

# Create sample slices from separate database columns to use as vector inputs
sample_scores <- head(marks_df$score, 5)
sample_years <- head(student_df$enrollment_year, 5)


# Use mapply to loop through both vectors element-by-element in parallel
performance_flags <- mapply(
  FUN = flag_student_performance,
  sample_scores,
  sample_years
)

print(performance_flags)
```

```
## [1] "Standard Performance" "Elite (2024 cohort)"  "High Performer"
## [4] "Elite (2024 cohort)"  "Standard Performance"
```